B 2.3.3 Looping Python

B 2.3.3 Construct programs that utilize looping structures to perform repeated actions Python

Loops are essential for repetitive tasks. Learn how for, while, and do-while loops (Java) or for and while loops (Python) work, and how to control them with conditional statements and operators. Explore examples ranging from basic iterations to nested loops.

Learning Objectives

- Understand the purpose and application of loops in programming.
- Differentiate between different loop types (for, while, do-while in Java).
- Explain how conditional statements and operators control loop execution.
- Implement various loop types to solve different programming problems.

Introduction

Python provides two primary loop types: for and while. These loops allow you to repeat a block of code multiple times, making your programs more efficient and concise.

for Loop

The for loop in Python is used to iterate over a sequence (like a list, tuple, string, or range):

```
for i in range(5):
  print("Iteration:", i)
# prints out the numbers 0 to 5.
```

You can also choose what point to start and finish the for loop. For example

This code prints "Iteration: " followed by the current value of i five times (0 to 4).

```
for i in range(2,5):
  print("Iteration:", i)

# prints out the numbers 2 to 4.
```

Iterating over lists and strings:

```
fruits = ["apple", "banana", "cherry"]

for fruit in fruits:
  print(fruit)

name = "Alice"
for letter in name:
  print(letter)
```

These examples demonstrate how to loop through elements in a list and characters in a string.

while Loop

The while loop continues to execute as long as its condition is True:

```
count = 0

while count < 5:
  print("Count:", count)
  count += 1
```

This code prints "Count: " followed by the current value of count until count reaches 5.

Infinite while loop (and how to avoid it):

```
# This is an infinite loop!
while True:
  print("This will print forever!")
```

To avoid infinite loops, ensure the loop condition eventually becomes False.

Conditional Statements and Operators in Loops

Conditional statements (if, elif, else) and operators (and, or, not, <, >, <=, >=, ==, !=) can be used within loop bodies to control their behaviour:

```
for i in range(1, 11):
  if i % 2 == 0:
    print(i, "is even.")
  else:
    print(i, "is odd.")
```

This loop iterates through numbers 1 to 10 and prints whether each number is even or odd.

Nested Loops

You can place a loop inside another loop to create a nested loop. This is useful for working with multi-dimensional data structures (like matrices or tables) or generating patterns.

```
# Print a multiplication table (up to 5 x 5)
for i in range(1, 6):  # Outer loop (rows)
  for j in range(1, 6):  # Inner loop (columns)
    print(i * j, end="\t")  # Print the product with a tab separator
  print()  # Move to the next line after each row
```

Explanation:

- **Outer loop:** The outer for loop iterates through the rows of the multiplication table (from 1 to 5).
- **Inner loop:** For each row (i), the inner for loop iterates through the columns (from 1 to 5).
- **Calculation and printing:** Inside the inner loop, the product of i and j is calculated and printed with a tab separator (end="\t") to keep the output organised in columns.
- **Newline:** After each row is printed, print() (without any arguments) moves the output to the next line.

Output:

```
1	2	3	4	5
2	4	6	8	10
3	6	9	12	15
4	8	12	16	20
5	10	15	20	25
```

Key Considerations:

- **Order of execution:** The inner loop completes all its iterations for each iteration of the outer loop.
- **Complexity:** Nested loops can increase the time complexity of your code, especially with large datasets.
- **Readability:** Use proper indentation to make nested loops easier to read and understand.

Nested loops provide a powerful way to handle complex iterative tasks and manipulate data in multiple dimensions.

Controlling Loop Flow with break and continue

Python provides two keywords, `break` and `continue`, to give you more control over the execution of your loops.

break statement:

The `break` statement allows you to exit a loop prematurely, even if the loop condition is still true. This is useful when you need to terminate the loop based on a specific condition that occurs within the loop body.

```
# Find a number in a list
numbers = [10, 25, 42, 58, 71]
target = 42

for number in numbers:
  if number == target:
    print("Target found!")
    break  # Exit the loop once the target is found
```

In this example, the loop iterates through the numbers list. When it encounters the target value (42), the break statement is executed, and the loop terminates immediately.

continue statement:

The continue statement allows you to skip the rest of the current iteration and proceed to the next iteration of the loop. This is helpful when you want to ignore certain elements or conditions within the loop.

```
# Print only odd numbers from 1 to 10
for i in range(1, 11):
  if i % 2 == 0:
    continue  # Skip even numbers
  print(i)
```

In this example, the loop iterates through numbers from 1 to 10. If the current number (i) is even, the continue statement skips the print(i) statement and moves to the next iteration.

By using break and continue, you can create more flexible and efficient loops that handle a wider range of scenarios.

Key Questions

What are the key differences between for and while loops in Python?

for loop: Designed for iterating over a sequence (like a list, tuple, string, or range). It automatically handles the iteration process, making it concise and less error-prone for these scenarios.
while loop: More general-purpose. It continues to execute as long as a given condition remains True. You need to manage the loop variable and ensure the condition eventually becomes False.

How can you prevent infinite loops when using while loops?

Ensure the loop condition eventually becomes False: Make sure the condition you're checking is modified within the loop body in a way that will eventually cause it to be evaluated as False.
Use a counter: If you need to iterate a specific number of times, use a counter variable and increment or decrement it within the loop to reach the termination condition.
Include a break condition: If the loop needs to terminate based on an event or condition other than a simple counter, use an if statement and the break keyword to exit the loop when necessary.

How do conditional statements influence the execution of code within a loop?

Conditional statements (if, elif, else) give you control over which parts of the code inside a loop are executed. This lets you:

Perform different actions: Execute different blocks of code based on conditions met during each iteration.
Skip iterations: Use continue to skip the rest of the current iteration and move to the next one if a condition is met.
Terminate the loop: Use break to exit the loop entirely when a specific condition occurs.

What are some real-world examples of where nested loops are used in Python programming?

Generating patterns: Creating visual patterns like stars, numbers, or characters in various shapes.
Working with matrices or tables: Performing operations on rows and columns of 2D data structures.
Game development: Implementing game logic, like moving characters on a grid or processing interactions between game objects.

Image processing: Manipulating pixels in an image, applying filters, or detecting edges.
Searching and sorting: Implementing algorithms to search for specific elements or sort data within nested data structures.

How can you use break and continue statements to alter the flow of a loop?

break: Immediately terminates the loop, regardless of the loop condition. Useful for exiting loops when a specific condition is met (e.g., finding an item in a list).
continue: Skips the remaining code in the current iteration and proceeds to the next iteration. Useful for ignoring certain elements or conditions within the loop (e.g., processing only even numbers in a list).